Universität Stuttgart
Master of Science

Basic Distributed Graph Algorithms

Marco Aiello

Broadcast and Convergecast

- **Broadcast:** a process p_i sends a message $< M >$ to all nodes of the system
- **Convergecast:** Collect information from all processes to a designated p_r
- Assumptions: nodes have only knowledge of direct neighbours
- *Graph:* $< V, E >$ where V is a set of vertices and $E = \{(v_i, v_j)$ with $v_i, v_j \in V\}$ a set of vertices pairs called edges
- $n = |V|$ order of the graph, $l = |E|$ size of the graph
- *Tree:* a connected graph without loops
- *Spanning tree:* a full subset of a graph that is a tree

Spanning tree construction: Flooding

Code for p_i with $1 \leq i \leq n$

parent, $= \perp$ children = $\emptyset$

if $p_i = p_r$ then send $< M >$ to all neighbours

Upon receiving $< M >$ from neighbour p_j :

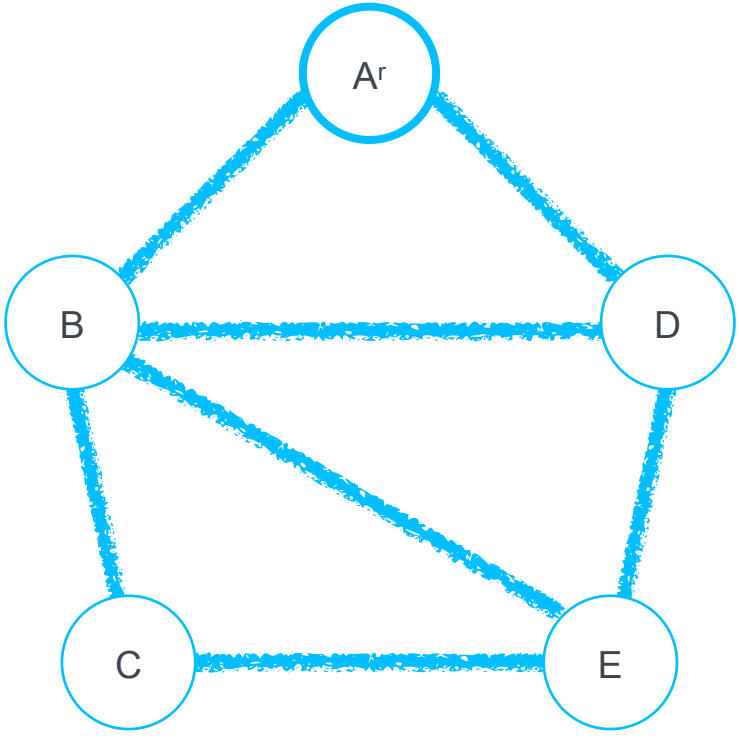
    if (parent $= \perp$) then
        parent = p_j ; send $<$ PARENT $>$ to p_j ;
        send $< M >$ to all neighbours except p_j ;
    else send $<$ ALREADY $>$ to p_j ;

Upon receiving $<$ PARENT $>$ from neighbour p_j :

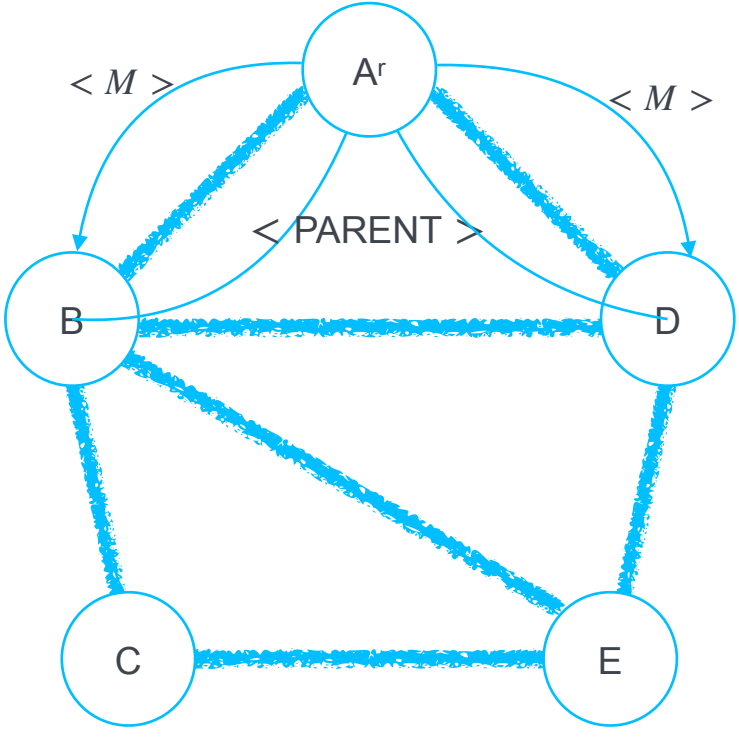
    children = children $\cup \{p_j\}$

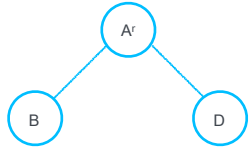
When received $<$ PARENT $>$ or $<$ ALREADY $>$ except parent, then terminate.

Example: Synchronous case

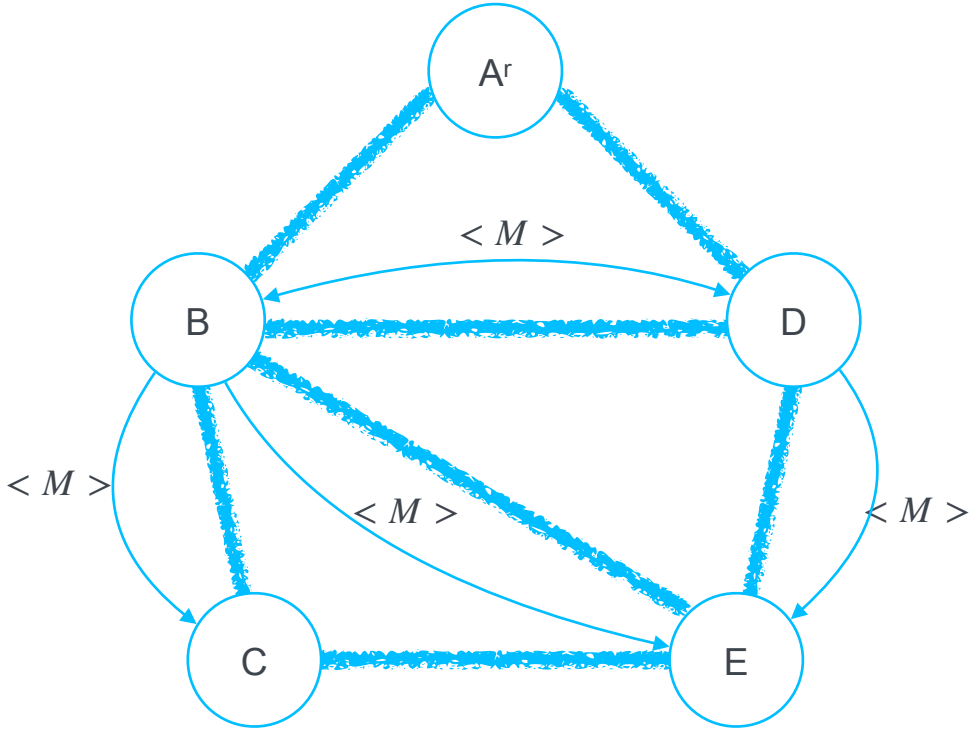


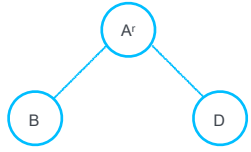
Example: Round 1



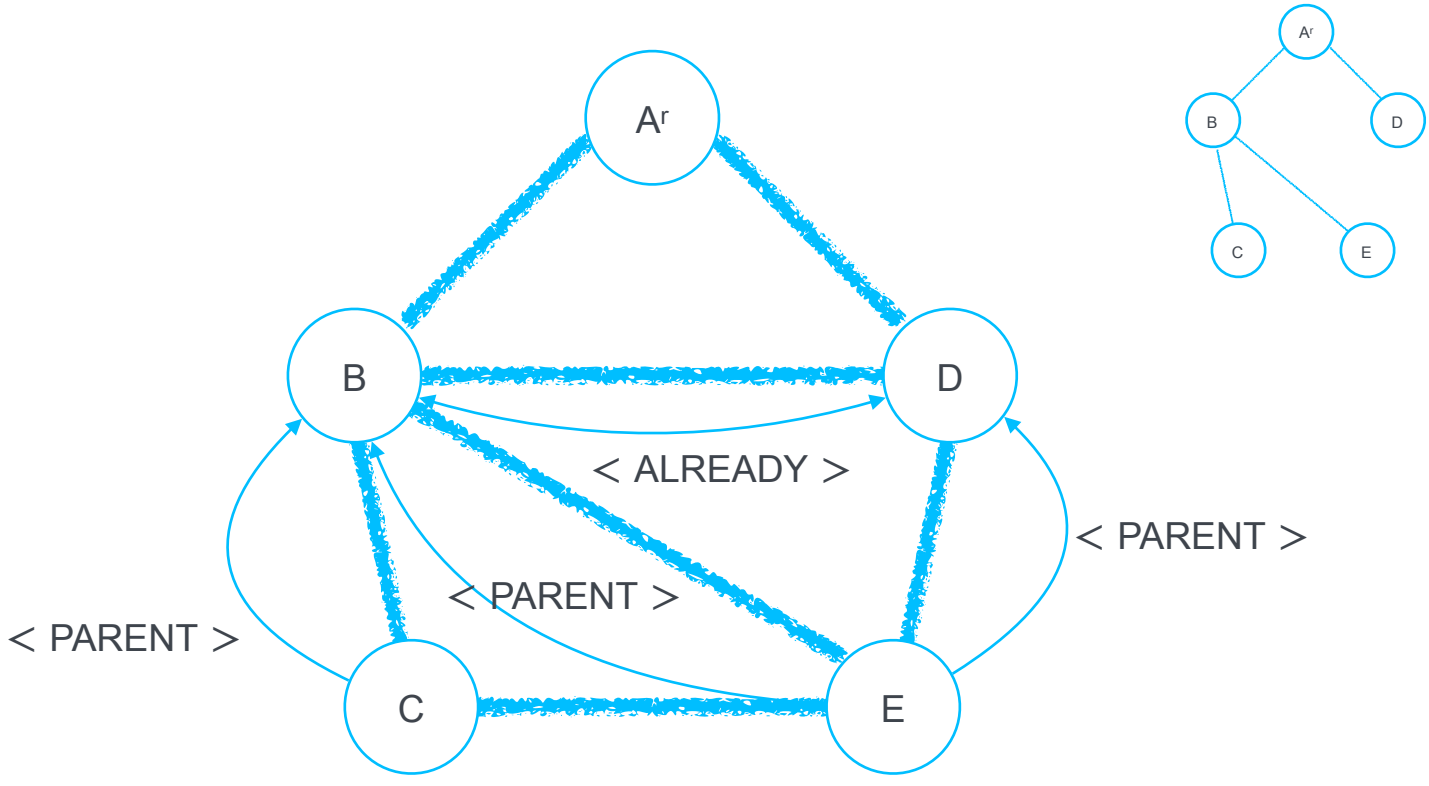


Example: Round 2

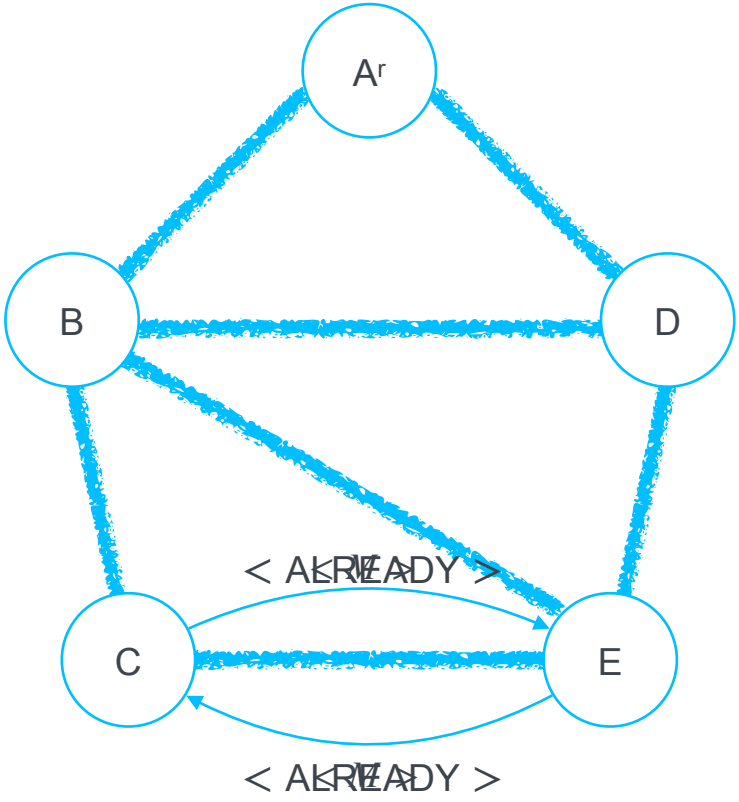


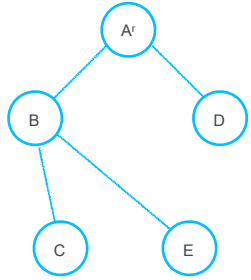


Example: Round 2



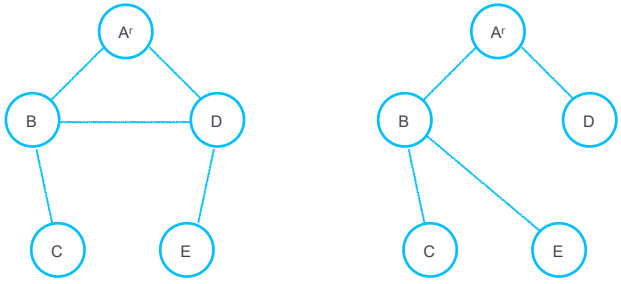
Example: Round 3





Discussion

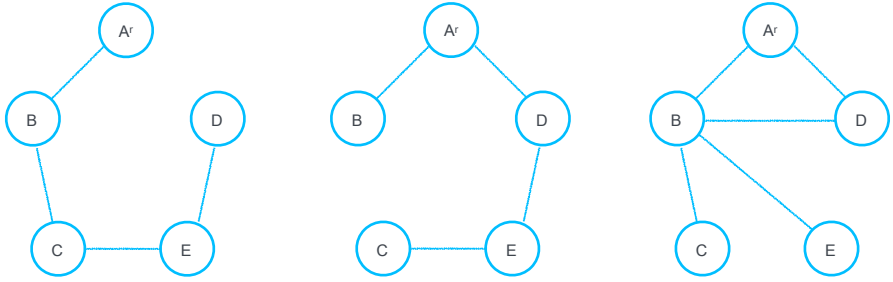
- We get a minimal depth spanning tree (breadth first or BFS)
- Both trees are possible



- *Message complexity:* <M> travels on all edges at most twice, i.e., $2l - (n - 1)$ plus the same amount of acknowledgement messages (< PARENT > < ALREADY >) so $O(l)$
- *Time complexity:* depends on the longest shortest path, i.e., the diameter d so $O(d)$

Asynchronous case

- Some channels communications could be slower than others, so no guarantees of a breadth first tree
- All these become possible trees



- To still obtain a BSF we can add a hop count to the messages that represents the distance from the root

Asynchronous case: BFS

Each p_i node has a variable my_dist, which represents the shortest path from the root that has been seen. Initially my_dist $= \infty$.

Upon receiving $< M, d >$ from p_j:

if my_dist > dist + 1 then parent= p_j, my_dist = dist + 1, send $<$ PARENT $>$ to p_j and $< M$, my_dist $>$ to the rest of the neighbours.

if my_dist = dist + 1 then send $<$ ALREADY $>$ to p_j.

if my_dist < dist + 1 do nothing

Complexity: $O(d)$ time and $O(nl)$ messages (each p_i reduces its my_dist at most $n - 1$ times)

Multiple roots

- There must be a unique identifier per node and a total order on the identifiers
- Every process keeps a variable my_root, initially set to -1, that represents the spanning tree root that will be finally formed.
- Each message is now $< M, r, d >$ representing the message of the algorithmic, the given root r for that message, and the distance d from it

Weighted graphs: minimum spanning tree

- Assume there is a function $f : E \rightarrow$ IR that assigns weights to edges
- A *Minimum Spanning Tree (MST)* is a spanning tree for which the sum of the weights from the source to all other nodes is minimal.
- *Synchronous GHS (Gallagher, Humblet and Spira)* algorithm: start with the trivial spanning forest of n nodes and no edges and repeatedly connect components along minimum-weight outgoing edges (MWOE).
- We assume weights are unique

Synchronous GHS (Gallagher, Humblet and Spira)

- **Lemma:** For any spanning forest $\{(N_i, L_i) \mid i = 1...k\}$ of a weighted undirected graph G, consider any component (N_j, L_j). Denote by λ_j the edge with the smallest weight among those that have exactly one endpoint in N_j. Then an MST for G that includes all edges in each L_j of the spanning forest must also include λ_j.

- *Proof Sketch:* If for any component (N_j, L_j) , instead of another outgoing edge is used, then the resulting tree cannot be an MST, because the replacement λ_j' by λ_j yields a lower cost tree. Thus, if we start with a forest all of whose edges are in the unique MST, then the MWOEs of all components are also in the MST.

Synchronous GHS (Gallagher, Humblet and Spira)

- Proceed in levels: the level k components constitute a spanning forest, where each component consists of a tree that is a subgraph of the MST.
- Each component, at every level, has a distinguished node, the leader. All nodes know their component's leader's id IID.
- Each p_i knows which of its incident edges belong to the component's MST.

Synchronous GHS (Gallagher, Humblet and Spira) Algorithm

1) Initially, Level 0 consists of n single nodes
2) To construct the $k+1$ level components, each Level k component identifies its MWOE through a broadcast-convergecast-broadcast sequence.
3) The component leader broadcasts a search request SEARCH_MWOE (IID) along the tree edges.
4) Upon receiving SEARCH_MWOE(IID), each p_i sends test messages along all non-tree edges to find out if the other end belongs to the same component (has the same leader).
5) Each p_i selects among all its incident edges that are outgoing from the component the one with the minimum weight local_MWOE(i, j).
6) The local_MWOE(i, j) information is convergecasted to the leader, taking minima along the way.
7) The leader obtains the overall minimum MWOE(localID, remoteID) and broadcasts ADD_MWOE(localID, remoteID) along the tree edges. The node whose id is localID marks edge (localID, remoteID) as belonging to the component's MST.

Synchronous GHS (Gallagher, Humblet and Spira) Algorithm

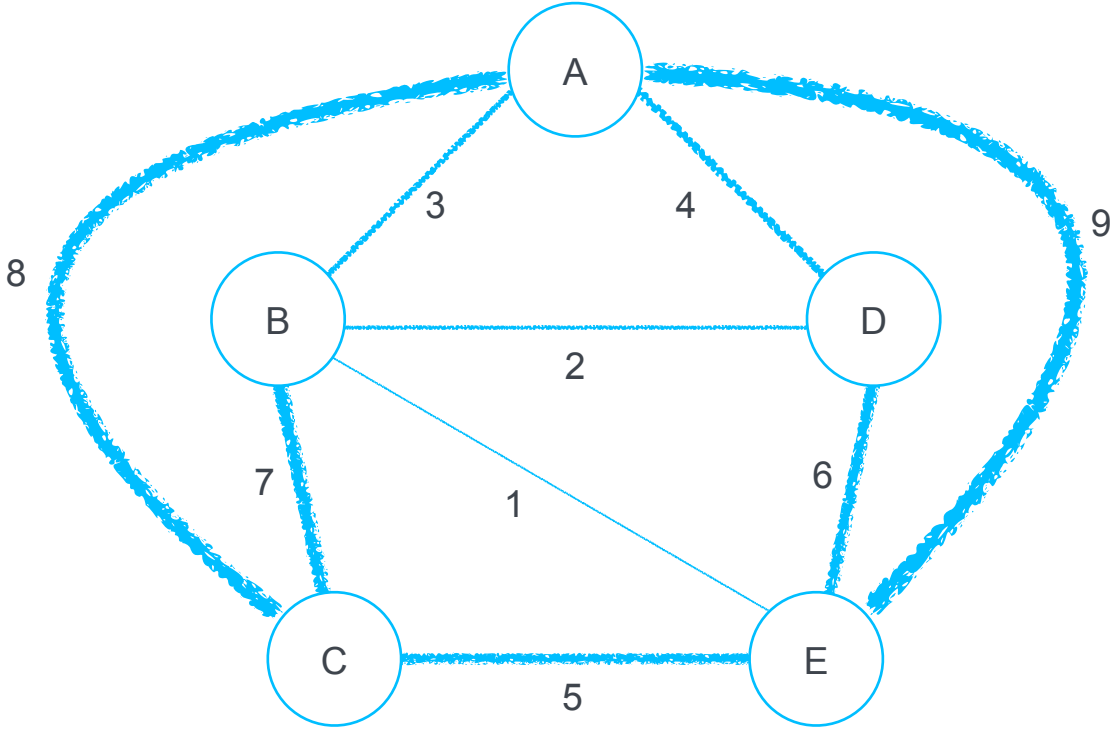
- **Lemma:** In each new component at Level $k+1$, there is exactly one edge that is the MWOE chosen by two fo the merged Level k components.
- *The proof is based on the fact that weights are distinct. If same weights are allowed, then use IDs as tie-breakers.*

8) If a MWOE is marked by both components on which it is incident, then the endpoint of the common MWOE with the larger id becomes the new leader: lID= max(localID, remoteID).

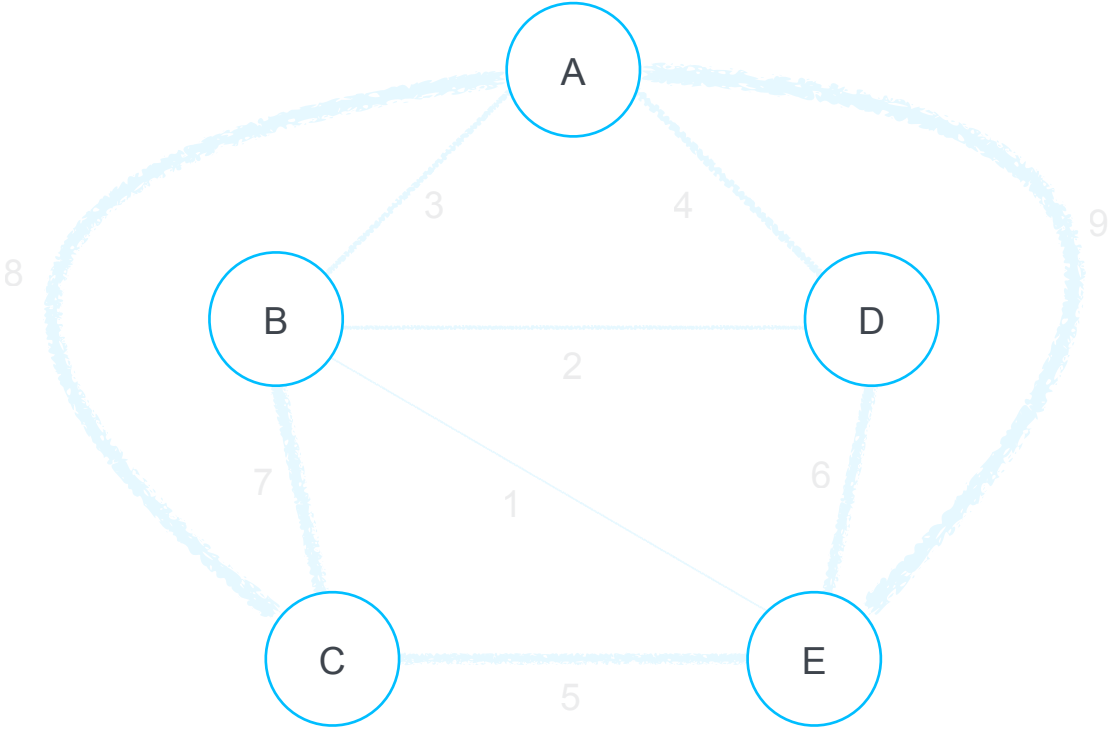
9) The new leader broadcasts a NEW_LEADER(lID) message along the MST of the newly formed component, so that all nodes in the $k+1$ round know their new leader.

10) The algorithm terminates when the leader learns that no MWOE can be found by any node in its component.

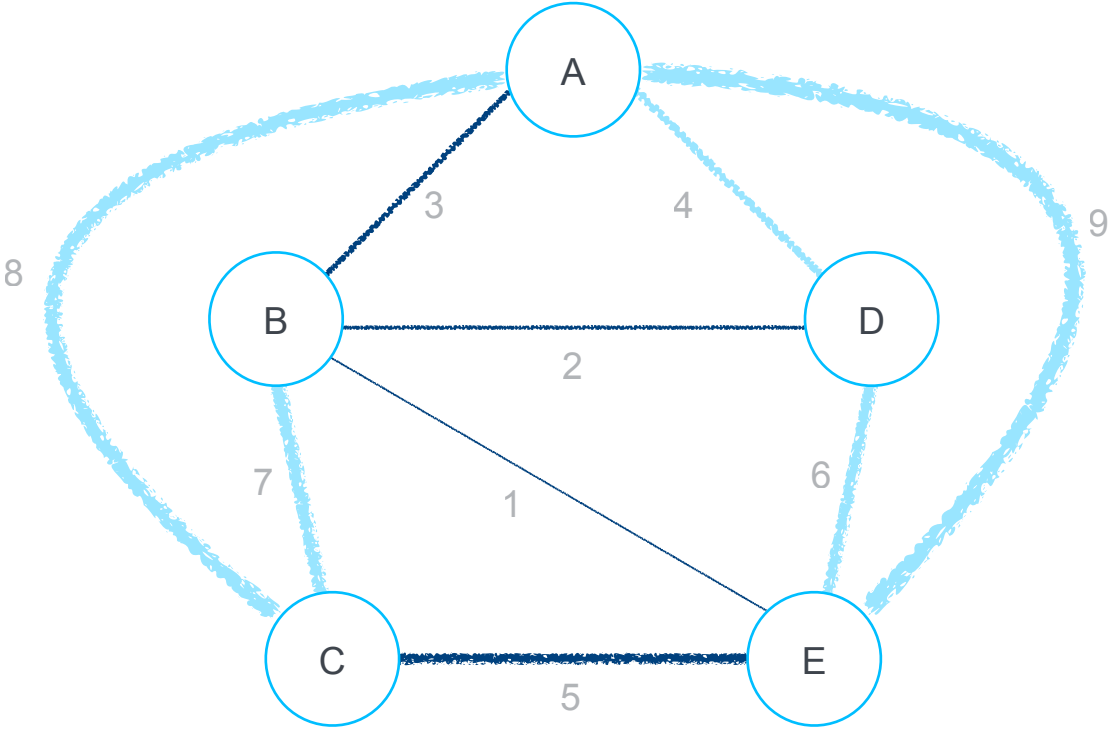
Synchronous GHS (Gallagher, Humblet and Spira) Example



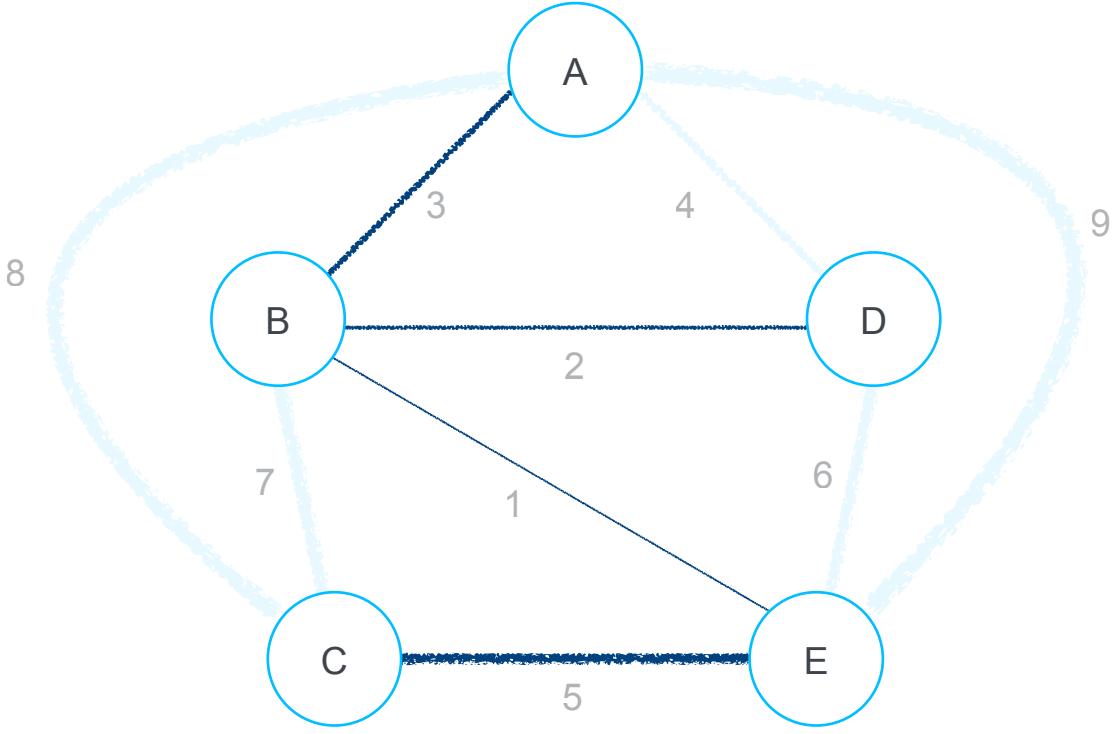
Synchronous GHS: Example: Level 0



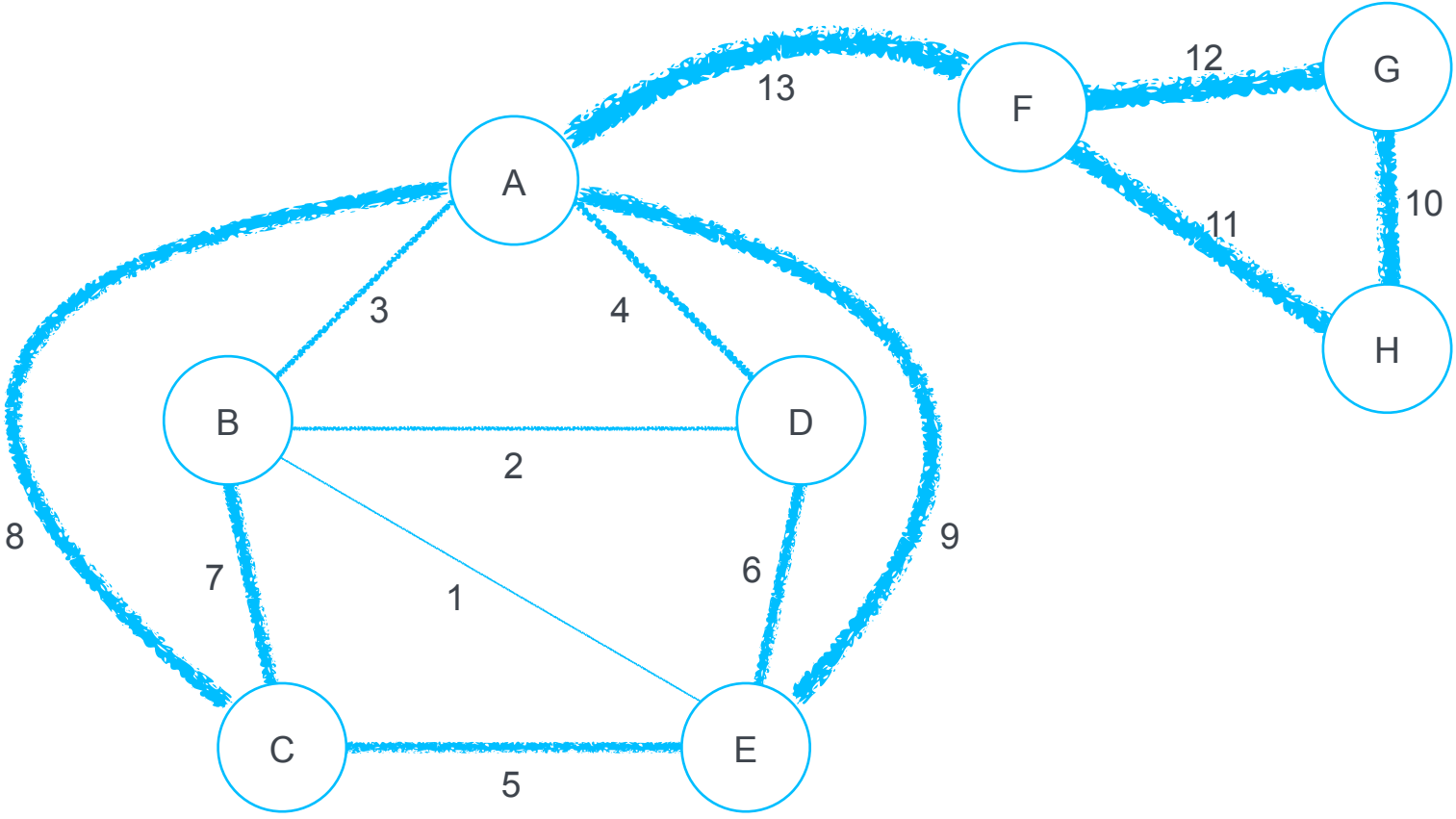
Synchronous GHS: Example: Level 1



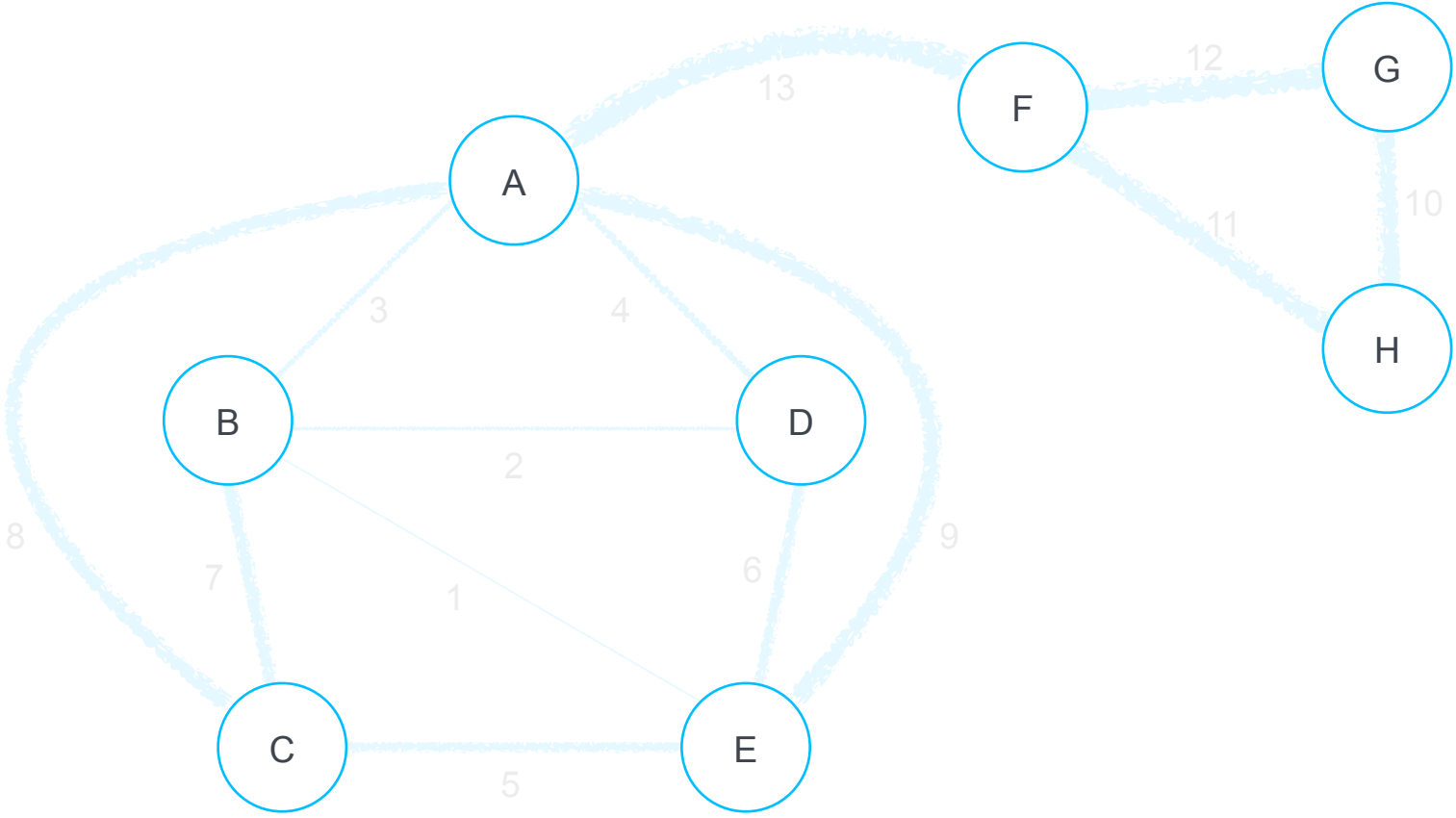
Synchronous GHS: Example: Termination



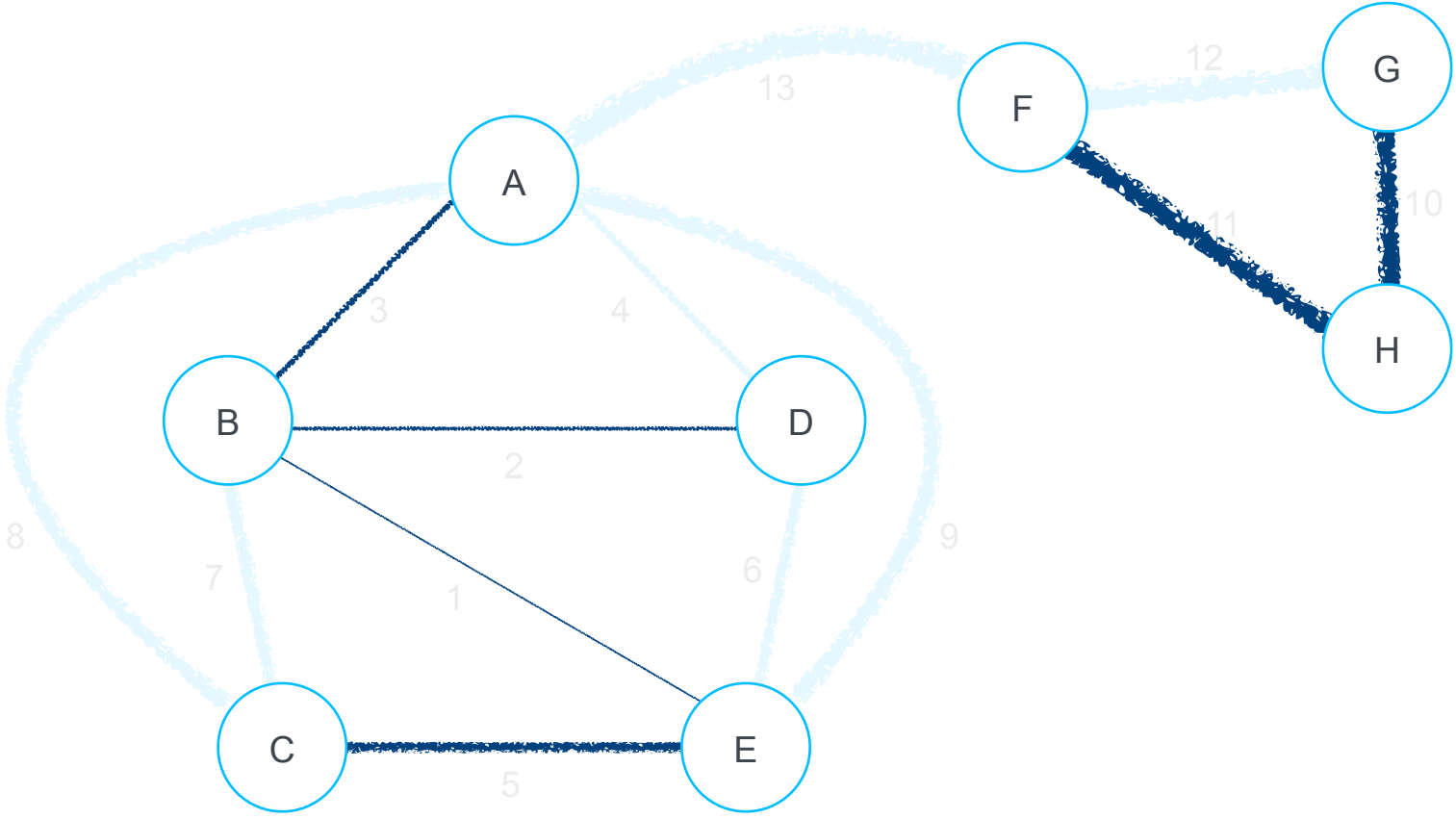
Synchronous GHS: Example 2



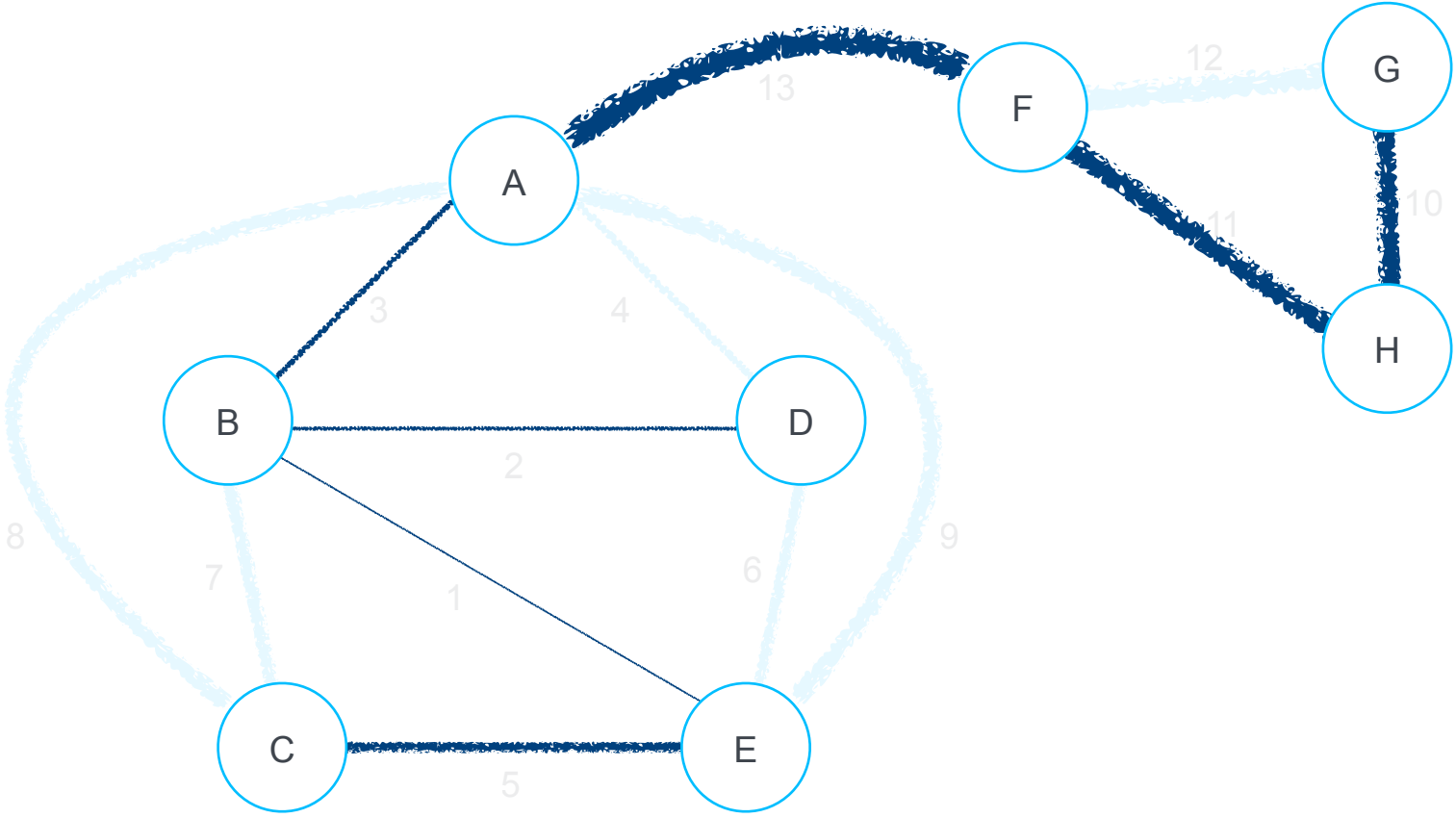
Synchronous GHS: Example 2: Level 0



Synchronous GHS: Example 2: Level 1



Synchronous GHS: Example 2: Level 2



Synchronous GHS (Gallagher, Humblet and Spira) Complexity

- A Level k component comprises at least 2^k nodes, so the number of levels is at most $\log n$. Each round consists of $O(n)$ messages that travel across a local MST.
- *Time complexity* is thus $O(n \log n)$
- *Message complexity:* At each level, $O(n)$ messages are sent for the broadcasts and convergecast along the tree edges, plus $O(l)$ to identify the local MWOE, so in total: $O((n + l)\log n)$
- If each node marks its incident edges that lead to a node of the same component as "rejected", then there is no need to test them again: each edge gets tested and rejected at most once, so $O(l)$ messages in total. In summary: $O(n \log n + l)$